Cloud Database Integration and API Implementation

Part II: Phases 1 to 3

Existing API Assessment, Database Design and Integration Architecture for the AegisNEO API on Neon

Name: Latagan, Jenny Joy F.
Course / Section: BSCS-SE CS301
Instructor: Sean Gono
Date: November 3, 2026

Table of Contents

1. Introduction 3
1.1 Purpose of this Document 3
1.2 Selected Database 3
1.3 Additional Requirements Given in Class 3
1.4 How the Evidence Was Collected 3
2. Phase 1: Analyze Your Existing API 4
2.1 Existing Application Structure 4
2.2 Existing Data Management 6
2.3 Existing API Functionality 8
2.4 Summary of Changes for Database Integration 16
2.5 Required Output of Phase 1 17
3. Phase 2: Design Your Database Structure 18
3.1 Data Entity Identification 18
3.2 Field Identification 18
3.3 Relationships and References 20
3.4 Database-Specific Design 20
3.5 Required Output of Phase 2 23
4. Phase 3: Plan the Integration Architecture 24
4.1 System Architecture 24
4.2 Request and Data Flow 25
4.3 Integration Approach 29
4.4 Security and Access Design 31
4.5 Required Output of Phase 3 32
5. References 33

1. Introduction

1.1 Purpose of this Document

This document covers Phases 1 to 3 of Part II: the assessment of the existing AegisNEO API, the design of its database structure, and the plan for connecting the API to the cloud database. No application code was changed during these phases. Implementation starts in Phase 4 and follows the design described here.

1.2 Selected Database

Part I selected **Neon** (serverless PostgreSQL): project AegisNEO in AWS Singapore (ap-southeast-1) on the Free plan, accessed through the pooled connection string with the limited role `aegis_app` (Part I, Section 6.1). Part II reuses this project. Its database `neondb` runs PostgreSQL 18.6 and still contains the test table from Part I (4 sample records), which is replaced by the design in Phase 2.

1.3 Additional Requirements Given in Class

Besides the activity guide, the following requirements were given during the class briefing. They are part of the design in Phases 2 and 3:

- All application data must come from the database. The API may no longer read the JSON file.
- The application must have a login screen with a username and password, so that the security of the API can be tested.
- There are two levels of access. A guest can see only a small part of the data (about 10 records). A logged-in user can see all records.
- Data must be persistent.
- The documentation must be clear and supported by screenshots.

1.4 How the Evidence Was Collected

The Phase 1 evidence was collected on 3 November 2026 by running the current API (version 1.1, commit `7e61ec8` on the `master` branch, which Vercel deploys) on a local machine with Uvicorn and sending requests with curl. The deployed site could not be reached from the network of the test machine, so the local run of the same code and data was used. Long responses are shortened in the figures; the full responses are saved in the repository under `docs/part2/evidence/phase1/`.

2. Phase 1: Analyze Your Existing API

2.1 Existing Application Structure

Purpose of the API

AegisNEO is a REST API about near-Earth objects, the asteroids whose orbits bring them close to Earth. Clients can browse, search and filter a catalog of asteroids, look up one asteroid, request random asteroids and read summary statistics. The API is used by the AegisNEO website and by two companion sites, Impactor (an impact simulator) and NEO Atlas (an asteroid star atlas).

Type of Information Managed

The API manages physical and approach data for asteroids: estimated size, NASA hazard classification, close-approach date, speed relative to Earth, miss distance and brightness. The catalog has 33,511 asteroids taken from the Kaggle dataset "Nearest Earth Objects (1910-2024)", which is based on NASA data. The source has no dates, so `close_approach_date` was generated once by `scripts/build_dataset.py` (a fixed date per asteroid). All other values come from the source.

Main Resources and Data Entities

The API has one entity, the **asteroid**. Its fields are defined by the Pydantic model `Asteroid` in `api/index.py`. There are no user accounts and no other entities.

Table 1. Fields of the asteroid entity

Field	Type	Example (Apophis)	Meaning
`neo_reference_id`	string	"2099942"	NASA reference number; identifies the asteroid
`name`	string	"99942 Apophis (2004 MN4)"	Asteroid designation
`estimated_diameter_km`	float	0.6539	Average of the minimum and maximum estimated diameter
`is_potentially_hazardous`	boolean	true	NASA potentially hazardous classification
`close_approach_date`	date	"1930-08-09"	Date of closest approach (generated, see above)
`relative_velocity_km_h`	float	14822.02	Speed relative to Earth in km/h
`miss_distance_km`	float	24400576.109	Distance from Earth at closest approach in km
`absolute_magnitude`	float	19.09	Brightness (H); a lower value means a larger object

Python Files, Modules and Components

Table 2. Files that make up the application

File	Role
`api/index.py`	The whole application (260 lines): configuration, Pydantic model, data loading, API key check, filtering and all endpoints.
`data/asteroids.json`	The asteroid catalog: 33,511 records, 8.2 MB. Read by the API at start-up.
`scripts/build_dataset.py`	One-time script that created `asteroids.json` from the Kaggle CSV. Not used while the API runs.
`data/raw/…csv`	Original Kaggle CSV (33.7 MB). Excluded from deployment by `.vercelignore`.
`requirements.txt`	Python packages: fastapi 0.141.1, uvicorn 0.47.0, pydantic 2.13.4.
`vercel.json`	Sends `/frontend/…` to the static website and every other path to `api/index.py`.
`frontend/`	Static website (`index.html`, `app.js`, `style.css`) that loads data from the API with `fetch()`.

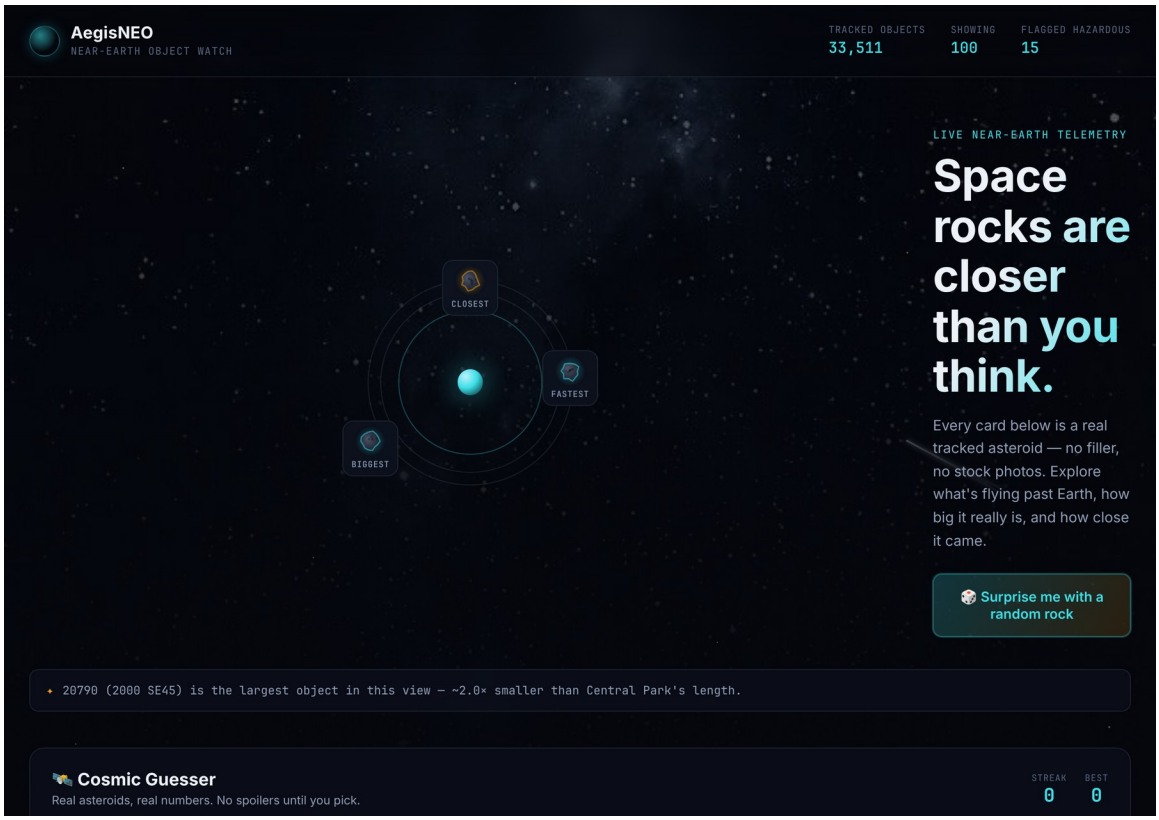


Figure 1. `website-before`: the AegisNEO website showing data from the current API (33,511 tracked objects)

Endpoints Currently Available

The API has six endpoints, all using the GET method: `/`, `/health`, `/api/v1/asteroids`, `/api/v1/asteroids/random`, `/api/v1/asteroids/{neo_reference_id}` and `/api/v1/stats`. Every `/api/v1/` endpoint requires an `x-api-key` header. Section 2.3 describes each endpoint in detail.

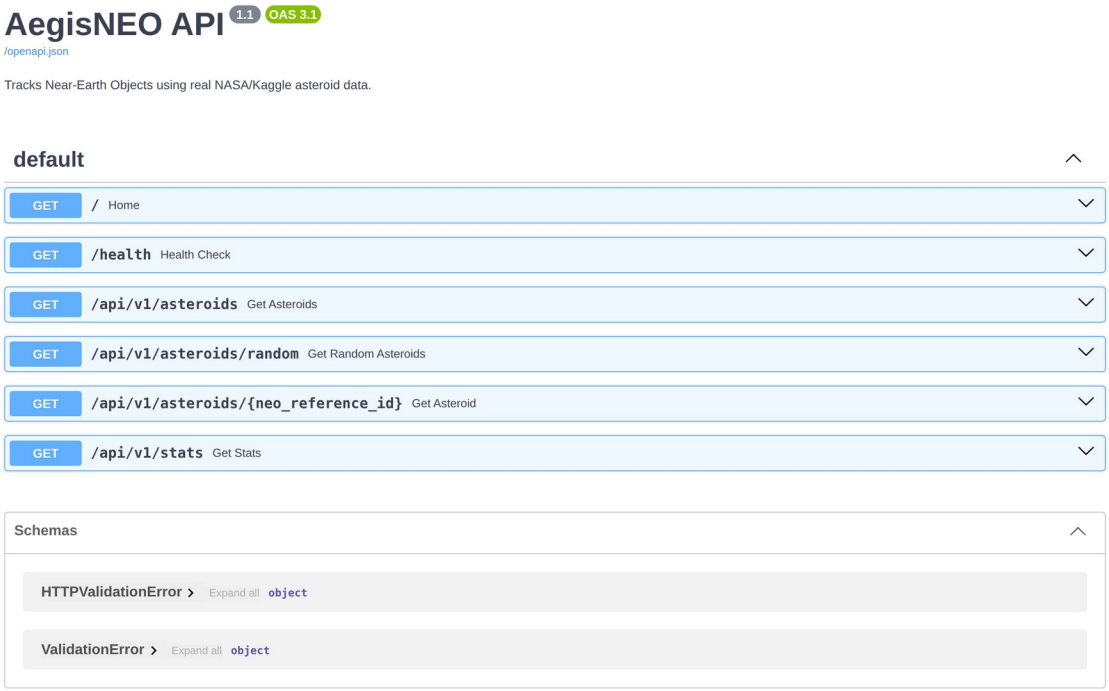


Figure 2. `swagger-before`: Swagger UI of the current API (version 1.1) with its six GET endpoints

2.2 Existing Data Management

Where the Records Are Stored

All records are stored in the file `data/asteroids.json`. The file is part of the GitHub repository and is uploaded with every Vercel deployment.

How the Records Are Held

The records are not typed into the Python code, but they are bundled with it. When the Python process starts (a cold start on Vercel), the file is read once, every record is validated with the `Asteroid` model, and the results are kept in memory in two structures: a list `asteroids` used for listing and filtering, and a dictionary `asteroids_by_id` keyed by `neo_reference_id` for single lookups. The summary statistics are computed once at the same time and kept in `CATALOG_STATS`.

```
# api/index.py, lines 70-77
DATA_PATH = Path(__file__).parent.parent / "data" / "asteroids.json"
with open(DATA_PATH, encoding="utf-8") as f:
    asteroids = json.load(f)

# Validate the full dataset against the schema when the application launches:
asteroids = [Asteroid(**a).model_dump(mode="json") for a in asteroids]

asteroids_by_id = {a["neo_reference_id"]: a for a in asteroids}
```

How the Application Retrieves and Modifies Records

- **List and search:** `filter_asteroids()` loops over all 33,511 records in Python and checks each filter. The result is sorted with `sorted()` and paged with list slicing (`results[offset:offset + limit]`).
- **One record:** a dictionary lookup, `asteroids_by_id.get(neo_reference_id)`.
- **Random records:** `random.sample()` over the filtered list.
- **Statistics:** the endpoint returns the dictionary computed at start-up.
- **Modification:** not possible. The API has no create, update or delete endpoints, and POST, PATCH and DELETE requests return 405 Method Not Allowed (Figure 11). The only way to change the data is to edit or regenerate the JSON file, commit it and deploy again.

```
# api/index.py, lines 140-158 (shortened)
def filter_asteroids(search=None, hazardous=None, min_diameter=None, max_diameter=None):
    ...
    results = []
    for asteroid in asteroids:
        if hazardous is not None and asteroid["is_potentially_hazardous"] != hazardous:
            continue
        if min_diameter is not None and asteroid["estimated_diameter_km"] < min_diameter:
            continue
        ...
        results.append(asteroid)
    return results
```

Limitations of the Current Approach

- **Read-only.** Records cannot be added, changed or removed through the API.
- **Changes would not persist.** Even if write endpoints were added, the changes would be lost. Vercel functions cannot write to their files, and each running instance keeps its own copy of the data in memory, which is discarded when the instance stops.
- **Data and code are tied together.** Any change to the data requires a new deployment.
- **Slow start and full scans.** Every cold start reads and validates the 8.2 MB file before answering, and every search loops over all records because there are no indexes.
- **Statistics only at start-up.** `CATALOG_STATS` would not reflect any change made while the API runs.
- **Weak access control.** The key check accepts a demo key written in `api/index.py` (`student-api-key-123`), and the website key is written in `frontend/app.js`. Both are stored in the GitHub repository, and either one gives full access. There are no user accounts, so a login screen and different access levels cannot be implemented.
- **Generated dates.** `close_approach_date` is not real data. This limitation stays in Part II and will be stated in the final report.

Changes Necessary to Support Persistent Database Storage

1. Create the tables designed in Phase 2 in Neon and import the 33,511 records once with a script. After the import, the API does not read the JSON file.
2. Add a data access module that runs SQL through the psycopg driver, using the pooled connection string stored in an environment variable.

3. Rewrite each read endpoint to query the database (filtering, sorting, paging, random selection and statistics in SQL), keeping the same response format for the existing website and companion sites.
4. Add endpoints to create, update and delete asteroids.
5. Add a users table and a login endpoint, limit guests to 10 featured records, and require login for create, update and delete.
6. Remove the keys written in the code, and keep every secret in Vercel environment variables.
7. Translate database errors (duplicate records, unavailable database) into clear HTTP responses.

2.3 Existing API Functionality

Table 3 lists every endpoint of the current API with its method, path, purpose, parameters, current response and whether it must change. The responses were recorded from the local run (Figures 3 to 11).

Table 3. Inventory of the existing endpoints

Method and path	Purpose	Parameters / request body	Current response	Modify?
GET `/`	Welcome message and list of endpoints	None	200: `message`, `total_tracked`, `endpoints`	Yes: count from the database
GET `/health`	Status check; no key needed	None	200: `status`, `service`, `version`, `timestamp`	Yes: add a database check
GET `/api/v1/asteroids`	List, search, filter, sort and page asteroids	Query: `search`, `hazardous`, `min_diameter`, `max_diameter`, `sort`, `order`, `limit` (1-100), `offset`. Header: `x-api-key`	200: `count`, `matched`, `total`, `offset`, `limit`, `asteroids`. 401 without a valid key; 422 for invalid parameters	Yes: SQL query; guests limited
GET `/api/v1/asteroids/random`	Random asteroids	Query: `count` (1-20), `hazardous`, `min_diameter`, `max_diameter`. Header: `x-api-key`	200: `count`, `matched`, `asteroids`	Yes: SQL query; guests limited
GET `/api/v1/asteroids/{neo_reference_id}`	One asteroid	Path: `neo_reference_id`. Header: `x-api-key`	200: the asteroid. 404: "Asteroid not found."	Yes: primary key lookup; guests limited
GET `/api/v1/stats`	Catalog summary	Header: `x-api-key`	200: `total`, `potentially_hazardous`, minimum, maximum and median values	Yes: SQL aggregates; login required

Create, update and delete operations do not exist. Requests with POST, PATCH or DELETE to the asteroid paths return 405 Method Not Allowed.

Recorded Responses of the Current API

Local run of the current API (v1.1) · GET / and GET /health · 2026-11-03 10:07 UTC

```
$ curl http://localhost:8000/

{
  "message": "Welcome to the AegisNEO API!",
  "total_tracked": 33511,
  "endpoints": [
    "/api/v1/asteroids",
    "/api/v1/asteroids?search=&hazardous=&min_diameter=&max_diameter=&sort=&order=&limit=&offset
=",
    "/api/v1/asteroids/random?count=",
    "/api/v1/asteroids/{neo_reference_id}",
    "/api/v1/stats"
  ]
}

HTTP status: 200

$ curl http://localhost:8000/health

{
  "status": "ok",
  "service": "AegisNEO API",
  "version": "1.1",
  "timestamp": "2026-11-03T10:08:03.107417Z"
}

HTTP status: 200
```

Figure 3. `p1-home-health`: GET / and GET /health

Current API · GET /api/v1/asteroids · 2026-11-03 10:07 UTC

```
$ curl -H 'x-api-key: student-api-key-123' http://localhost:8000/api/v1/asteroids

{
  "count": 100,
  "matched": 33511,
  "total": 33511,
  "offset": 0,
  "limit": 100,
  "asteroids": [
    {
      "neo_reference_id": "2162117",
      "name": "162117 (1998 SD15)",
      "estimated_diameter_km": 0.6391,
      "is_potentially_hazardous": false,
      "close_approach_date": "1925-01-22",
      "relative_velocity_km_h": 71745.401,
      "miss_distance_km": 58143623.319,
      "absolute_magnitude": 19.14
    },
    {
      "neo_reference_id": "2349507",
      "name": "349507 (2008 QY)",
      "estimated_diameter_km": 0.8581,
      "is_potentially_hazardous": true,
      "close_approach_date": "2008-03-02",
      "relative_velocity_km_h": 109949.757,
      "miss_distance_km": 55801047.818,
      "absolute_magnitude": 18.5
    },
    # 98 more asteroid objects not shown in this figure
  ]
}

HTTP status: 200
```

Figure 4. `p1 - list`: GET /api/v1/asteroids returns the first 100 of 33,511 records

Current API · search and filters · 2026-11-03 10:07 UTC

```
$ curl -H 'x-api-key: student-api-key-123' 'http://localhost:8000/api/v1/asteroids?search=apophis'

{
  "count": 1,
  "matched": 1,
  "total": 33511,
  "offset": 0,
  "limit": 100,
  "asteroids": [
    {
      "neo_reference_id": "2099942",
      "name": "99942 Apophis (2004 MN4)",
      "estimated_diameter_km": 0.6539,
      "is_potentially_hazardous": true,
      "close_approach_date": "1930-08-09",
      "relative_velocity_km_h": 14822.02,
      "miss_distance_km": 24400576.109,
      "absolute_magnitude": 19.09
    }
  ],
  "query": "apophis"
}

HTTP status: 200

$ curl -H 'x-api-key: student-api-key-123' 'http://localhost:8000/api/v1/asteroids?hazardous=true&min_diameter=1&sort=diameter&order=desc&limit=3'

{
  "count": 3,
  "matched": 219,
  "total": 33511,
  "offset": 0,
  "limit": 3,
  "asteroids": [
    {
      "neo_reference_id": "2003122",
      "name": "3122 Florence (1981 ET3)",
      "estimated_diameter_km": 6.5094,
      "is_potentially_hazardous": true,
      "close_approach_date": "1936-12-05",
      "relative_velocity_km_h": 73048.503,
      "miss_distance_km": 58055238.369,
      "absolute_magnitude": 14.1
    },
    {
      "neo_reference_id": "2004183",
      "name": "4183 Cuno (1959 LM)",
      "estimated_diameter_km": 6.3029,
      "is_potentially_hazardous": true,
      "close_approach_date": "2009-11-22",
      "relative_velocity_km_h": 53773.404,
      "miss_distance_km": 42214827.89,
      "absolute_magnitude": 14.17
    },
    # 1 more asteroid object not shown in this figure
  ]
}

HTTP status: 200
```

Figure 5. `p1-search-filter`: name search, and the hazard and diameter filters with sorting

Current API · GET /api/v1/asteroids/random · 2026-11-03 10:07 UTC

```
$ curl -H 'x-api-key: student-api-key-123' 'http://localhost:8000/api/v1/asteroids/random?count=
2'

{
  "count": 2,
  "matched": 33511,
  "asteroids": [
    {
      "neo_reference_id": "3599866",
      "name": "(2012 DK31)",
      "estimated_diameter_km": 0.1728,
      "is_potentially_hazardous": true,
      "close_approach_date": "2002-08-09",
      "relative_velocity_km_h": 45641.899,
      "miss_distance_km": 15573664.014,
      "absolute_magnitude": 21.98
    },
    {
      "neo_reference_id": "3661270",
      "name": "(2014 CQ)",
      "estimated_diameter_km": 0.1131,
      "is_potentially_hazardous": false,
      "close_approach_date": "2018-07-26",
      "relative_velocity_km_h": 71903.118,
      "miss_distance_km": 18474291.495,
      "absolute_magnitude": 22.9
    }
  ]
}

HTTP status: 200
```

Figure 6. `p1 - random`: GET /api/v1/asteroids/random?count=2



Figure 7. `p1-get-one`: one asteroid by ID, and a request for an ID that does not exist (404)

Current API · GET /api/v1/stats · 2026-11-03 10:07 UTC

```
$ curl -H 'x-api-key: student-api-key-123' http://localhost:8000/api/v1/stats

{
  "total": 33511,
  "potentially_hazardous": 2493,
  "estimated_diameter_km": {
    "min": 0.0008,
    "max": 60.7495,
    "median": 0.0783
  },
  "relative_velocity_km_h": {
    "min": 439.939,
    "max": 273704.636,
    "median": 47145.341
  },
  "miss_distance_km": {
    "min": 9316.925,
    "max": 74792800.949,
    "median": 35229281.508
  },
  "close_approach_date": {
    "min": "1910-01-01",
    "max": "2024-12-29"
  }
}

HTTP status: 200
```

Figure 8. `p1-stats`: GET /api/v1/stats, computed once at start-up

Current API · requests without a valid x-api-key · 2026-11-03 10:07 UTC

```
$ curl http://localhost:8000/api/v1/asteroids

{
  "detail": "Invalid or missing API key."
}

HTTP status: 401

$ curl -H 'x-api-key: wrong' http://localhost:8000/api/v1/asteroids

{
  "detail": "Invalid or missing API key."
}

HTTP status: 401
```

Figure 9. `p1-auth`: requests with no key and with a wrong key are rejected (401)

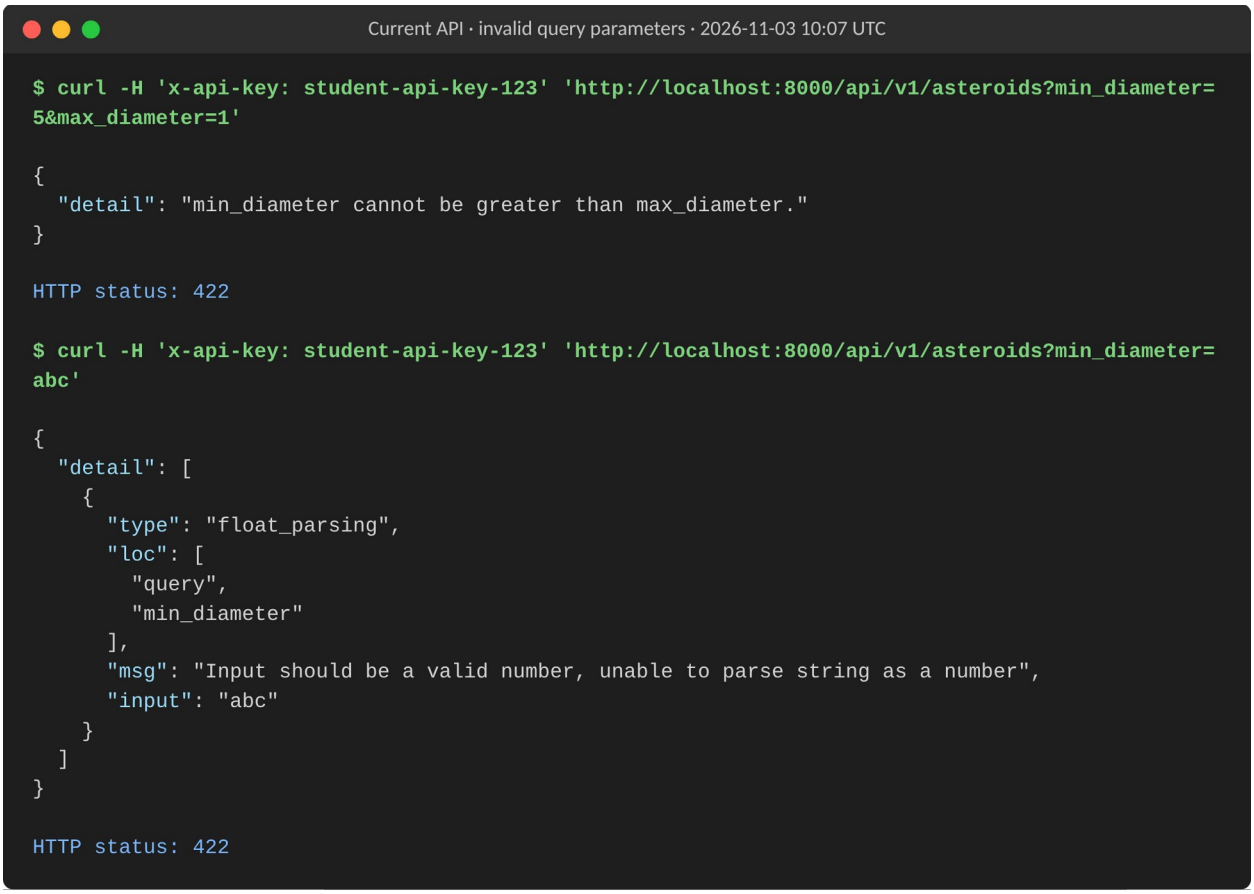


Figure 10. `p1-validation`: invalid query parameters are rejected (422)

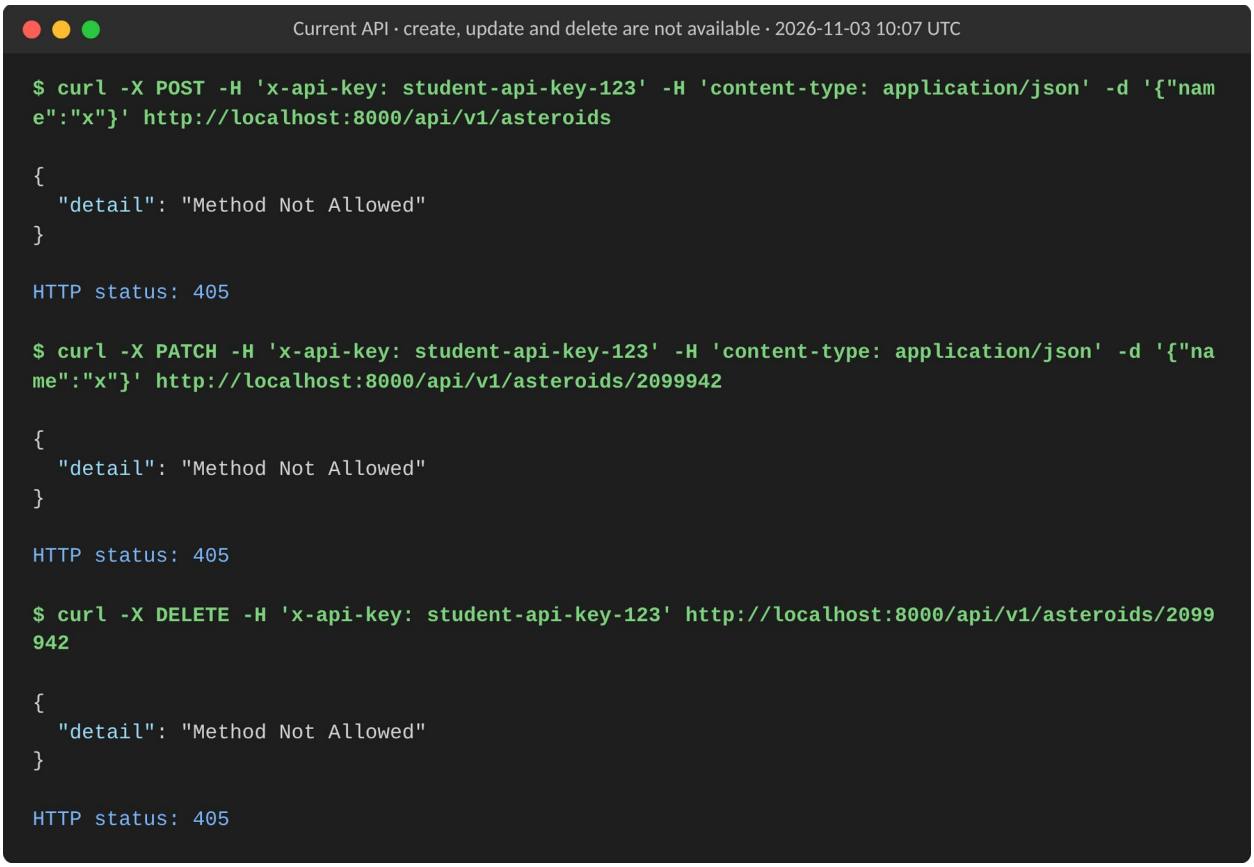


Figure 11. `p1-no-write`: POST, PATCH and DELETE are not supported (405)

2.4 Summary of Changes for Database Integration

Table 4 shows what will happen to each original endpoint in Part II and which endpoints will be added.

Table 4. Original and planned endpoints

Endpoint	Status in Part II	Change
GET `/`	Modified	`total_tracked` comes from a database count
GET `/health`	Modified	Also runs `SELECT 1`; returns 503 if the database cannot be reached
GET `/api/v1/asteroids`	Modified	SQL query with the same filters, plus a date range filter; guests see featured records only
GET `/api/v1/asteroids/random`	Modified	Random selection in SQL; guests get featured records only
GET `/api/v1/asteroids/{neo_reference_id}`	Modified	Primary key lookup; guests can open featured records only
GET `/api/v1/stats`	Modified	SQL aggregates computed on each request; login or partner key required
POST `/api/v1/asteroids`	New	Create an asteroid (login required)

Endpoint	Status in Part II	Change
PATCH `/api/v1/asteroids/{neo_reference_id}`	New	Partial update of an asteroid (login required)
DELETE `/api/v1/asteroids/{neo_reference_id}`	New	Delete an asteroid (login required)
POST `/api/v1/auth/login`	New	Check username and password; return an access token
GET `/api/v1/auth/me`	New	Return the logged-in user (used by the login screen)

2.5 Required Output of Phase 1

Table 5. Phase 1 required output

Required by the activity	Where it is answered
Application overview	Section 2.1
Existing data management	Section 2.2
Endpoint inventory	Section 2.3, Table 3 and Figures 3 to 11
Explanation of the changes required for database integration	Sections 2.2 and 2.4, Table 4

3. Phase 2: Design Your Database Structure

3.1 Data Entity Identification

The database has two entities:

- **Asteroid** (table `asteroids`): the main resource of the API, one row per asteroid. The import fills it with the 33,511 records of the current catalog.
- **User** (table `users`): an account that can log in. It is needed for the login screen and for the rule that guests see only part of the data. Accounts are created by the developer with a script. There is no public sign-up page, because a logged-in user can change the catalog.

The following were considered and not used:

- A separate table of close approaches (one asteroid, many approaches). The Kaggle source repeats rows for the same asteroid, but the catalog keeps one row per asteroid and its dates are generated, so a second table would contain no real extra information.
- A roles table. There are only two levels of access, and a guest is simply a request without a login, so no role needs to be stored.
- A table of featured asteroids. Whether an asteroid is shown to guests is a yes-or-no property of that asteroid, so one boolean column is enough.

3.2 Field Identification

Table 6 defines the fields of the `asteroids` table. The first eight fields are the existing API fields; the last four are new. Clients set the first nine fields; `created_by`, `created_at` and `updated_at` are set by the API and the database.

Table 6. Fields of the asteroids table

Field	Data type	Purpose	Required	Unique	Validation rules
`neo_reference_id`	text	NASA reference number; primary key	Yes	Yes	Digits only, 1 to 10 characters. Cannot be changed after creation.
`name`	text	Asteroid designation	Yes	Yes, ignoring case	1 to 100 characters after trimming spaces
`estimated_diameter_km`	double precision	Estimated diameter in km	Yes	No	Greater than 0 and at most 1,000
`is_potentially_hazardous`	boolean	NASA hazard classification	Yes	No	true or false; default false
`close_approach_date`	date	Date of closest approach	Yes	No	Valid date from 1900-01-01 to 2200-12-31
`relative_velocity_km_h`	double precision	Speed relative to Earth in km/h	Yes	No	0 to 1,000,000
`miss_distance_km`	double precision	Distance at closest approach in km	Yes	No	0 to 1,000,000,000

Field	Data type	Purpose	Required	Unique	Validation rules
absolute_magnitude	double precision	Brightness (H)	Yes	No	0 to 40
is_featured	boolean	Shown to guests	No (default false)	No	At most 10 asteroids can be featured
created_by	integer	User who created the record	No	No	Must match an existing users.id; empty for imported records
created_at	timestamptz	When the record was added	Automatic	No	Default now(); not accepted from clients
updated_at	timestamptz	When the record was last changed	Automatic	No	Set to now() on every update

The ranges were chosen from the current data, which has diameters from 0.0008 to 60.75 km, speeds up to 273,705 km/h, miss distances up to 74.8 million km and magnitudes from 9.25 to 33.58, leaving room for new records. The upper limits also stop the special floating-point values NaN and Infinity, which PostgreSQL would otherwise accept in a `double precision` column (NaN is treated as greater than every number, so a rule such as `>= 0` alone does not reject it).

Table 7. Fields of the users table

Field	Data type	Purpose	Required	Unique	Validation rules
id	integer (identity)	Primary key, generated by the database	Automatic	Yes	Cannot be set by clients
username	text	Name used to log in	Yes	Yes	3 to 30 characters: lowercase letters, digits and underscore
password_hash	text	bcrypt hash of the password	Yes	No	60-character bcrypt hash. The password itself is never stored and the hash is never returned by the API.
created_at	timestamptz	When the account was created	Automatic	No	Default now()

Passwords must be at least 8 characters long. This rule is checked by the account script before the password is hashed, because the database only receives the hash.

3.3 Relationships and References

- **One-to-many: users to asteroids.** A user can create many asteroids, and each asteroid was created by at most one user. The column `asteroids.created_by` is a foreign key to `users.id`. It is empty for the 33,511 imported records, which were not created through the API.
- **On delete:** `ON DELETE SET NULL`. If an account is removed, the asteroids it created stay in the catalog and lose only the reference.
- **One-to-one and many-to-many:** none. The API stores no information that links many users to many asteroids (for example, favourites).

A simple structure is appropriate because each asteroid is a self-contained record: all of its information fits in one row with fixed fields, and no record depends on another asteroid.

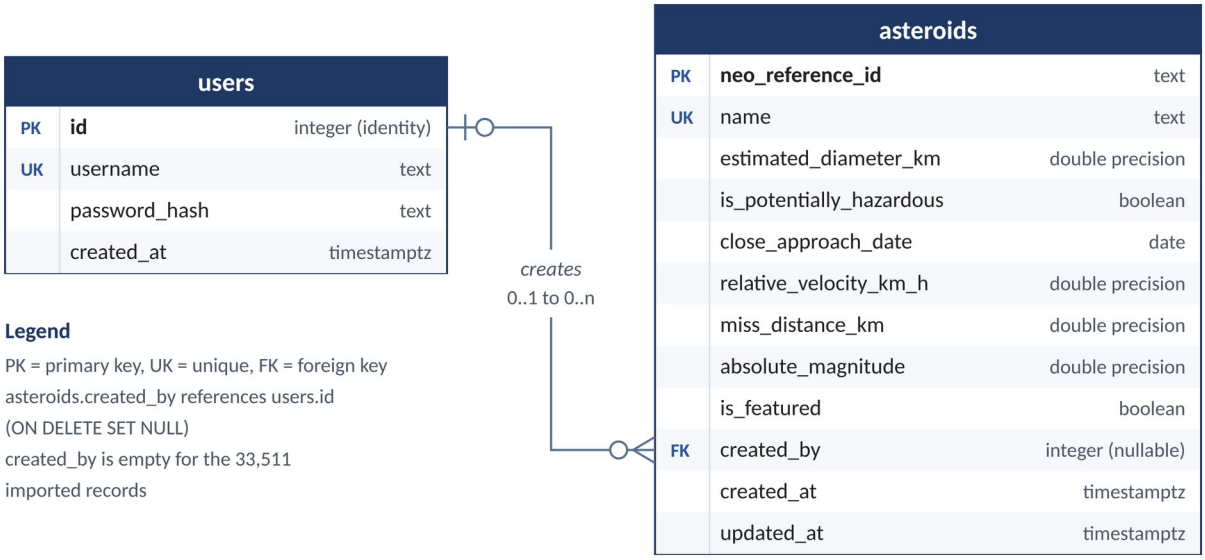


Figure 12. `erd`: entity-relationship diagram of the AegisNEO database

3.4 Database-Specific Design

Neon is PostgreSQL, so the design uses tables, columns, primary keys, a foreign key and constraints. This fits the data: every asteroid has the same fields with fixed types, and the API needs filtering, sorting, paging, counting and aggregates, which are standard SQL operations. The validation rules from Section 3.2 are written as database constraints, so invalid data is rejected even if the API code has a mistake. Part I showed that Neon rejects such data with error codes 23514 (check), 23502 (not null), 23505 (unique) and 22P02 (invalid value).

Primary Keys

- `asteroids.neo_reference_id` is a natural key. It is NASA's identifier, is already used in the API paths and by the existing clients, and never changes.
- `users.id` is a generated number, so a username could be changed without breaking the foreign key.

Indexes

Table 8. Indexes

Index	Columns	Used by	Reason
`asteroids_pkey`	`neo_reference_id`	Get, update and delete one asteroid	Created automatically with the primary key
`asteroids_name_lower_key` (unique)	`lower(name)`	Create and update	Prevents the same name with different letter case
`asteroids_name_trgm_idx` (GIN)	`name` (trigram)	Text search	Lets `ILIKE '%text%'` use an index instead of reading every row (extension `pg_trgm`)
`asteroids_diameter_idx`	`estimated_diameter_km`	Diameter filter and sort	Range conditions and ordering
`asteroids_date_idx`	`close_approach_date`	Date range filter and sort	Range conditions and ordering
`asteroids_featured_idx` (partial)	`name` where `is_featured`	All guest requests	Small index of the 10 featured rows
`asteroids_created_by_idx`	`created_by`	Foreign key	PostgreSQL does not index foreign keys automatically
`users_username_key` (unique)	`username`	Login	Created by the UNIQUE constraint

The hazard flag, speed, miss distance and magnitude are not indexed. The hazard flag has only two values, and sorting 33,511 rows by the other columns is fast without an index. The query plans will be checked with EXPLAIN in Phase 4.

Roles and Privileges

Table 9. Database roles

Role	Used by	Privileges
`neondb_owner`	Setup scripts on the developer's machine only	Owner of the database: creates tables, indexes and the extension; runs the data import and the account script. Never stored in Vercel.
`aegis_app`	The API on Vercel	`asteroids`: SELECT, INSERT, UPDATE, DELETE. `users`: SELECT only. Cannot create, change or drop tables, and cannot create accounts.

Rules Kept in the API

- **At most 10 featured asteroids.** A CHECK constraint can only look at one row, so this rule is checked by the API, which counts the featured rows in the same transaction before setting `is_featured` to true.
- **updated_at.** The API sets `updated_at = now()` in every UPDATE statement.

Schema (SQL)

The schema below will be run as `neondb_owner` in Phase 4, after the Part I test table is dropped. It is stored in the repository as `db/schema.sql`.

```
create extension if not exists pg_trgm;

create table public.users (
  id             integer generated always as identity primary key,
  username       text not null unique
                 check (username ~ '^[a-z0-9_]{3,30}$'),
  password_hash  text not null check (length(password_hash) = 60),
  created_at     timestamptz not null default now()
);

create table public.asteroids (
  neo_reference_id          text primary key
                            check (neo_reference_id ~ '^[0-9]{1,10}$'),
  name                      text not null
                            check (length(btrim(name)) between 1 and 100),
  estimated_diameter_km     double precision not null
                            check (estimated_diameter_km > 0
                                   and estimated_diameter_km <= 1000),
  is_potentially_hazardous  boolean not null default false,
  close_approach_date       date not null
                            check (close_approach_date
                                   between '1900-01-01' and '2200-12-31'),
  relative_velocity_km_h    double precision not null
                            check (relative_velocity_km_h between 0 and 1000000),
  miss_distance_km          double precision not null
                            check (miss_distance_km between 0 and 1000000000),
  absolute_magnitude        double precision not null
                            check (absolute_magnitude between 0 and 40),
  is_featured               boolean not null default false,
  created_by                integer
                            references public.users (id) on delete set null,
  created_at                timestamptz not null default now(),
  updated_at                timestamptz not null default now()
);

create unique index asteroids_name_lower_key on public.asteroids (lower(name));
create index asteroids_name_trgm_idx  on public.asteroids using gin (name gin_trgm_ops);
create index asteroids_diameter_idx   on public.asteroids (estimated_diameter_km);
create index asteroids_date_idx       on public.asteroids (close_approach_date);
create index asteroids_featured_idx   on public.asteroids (name) where is_featured;
create index asteroids_created_by_idx on public.asteroids (created_by);

grant select, insert, update, delete on public.asteroids to aegis_app;
grant select on public.users to aegis_app;
```

Initial Data

- All 33,511 asteroids from `data/asteroids.json` are imported once by `scripts/import_asteroids.py`, run as `neondb_owner`.

- Ten well-known asteroids are marked as featured for guests: Apophis, Bennu, Eros, Ganymed, Icarus, Toutatis, Didymos, Phaethon, Itokawa and Ryugu. Eight are classified as potentially hazardous and two are not, so the hazard filter also gives a visible result for guests.
- Demonstration accounts are created by `scripts/create_user.py`, which hashes the password with bcrypt before storing it.

3.5 Required Output of Phase 2

Table 10. Phase 2 required output

Required by the activity	Where it is answered
Entity description	Section 3.1
Field definitions	Section 3.2, Tables 6 and 7
Relationships or references	Section 3.3
Diagram of the proposed structure	Figure 12
Design adapted to the selected database	Section 3.4

4. Phase 3: Plan the Integration Architecture

4.1 System Architecture

Figure 13 shows the components and how they connect. Clients talk only to the API over HTTPS. The API runs on Vercel as a Python serverless function and is the only component that connects to Neon.

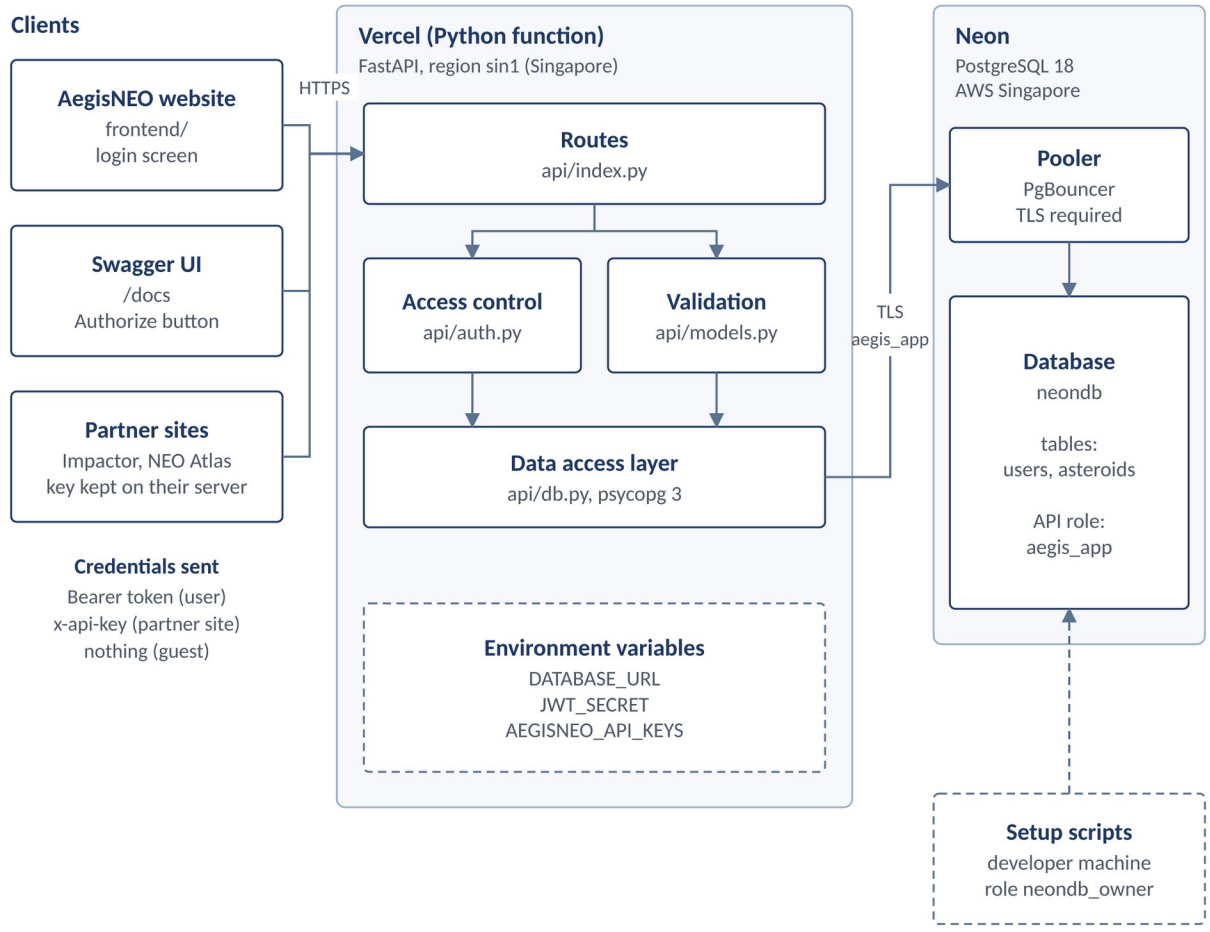


Figure 13. `architecture`: system architecture of the database-integrated AegisNEO API

Table 11. Components and responsibilities

Component	Technology	Responsibility
Client: AegisNEO website	HTML, CSS, JavaScript (`frontend/`)	Login screen with username and password, or "Continue as guest". Keeps the access token for the browser session and sends it with each request.
Client: Swagger UI	FastAPI `/docs`	Test every endpoint. The Authorize button logs in with a username and password.
Client: partner sites	Impactor, NEO Atlas	Read the full catalog through their own server-side proxies, each with a secret key. They cannot change data.
REST API	FastAPI on Vercel	Routes, HTTP methods, status codes and the JSON response format (`api/index.py`).

Component	Technology	Responsibility
Application logic	Python	Identify the caller, apply guest limits, validate data, enforce the featured limit and translate errors (`api/auth.py`, `api/models.py`).
Database connection	psycopg 3	Open a TLS connection through the Neon pooler and run parameterised SQL (`api/db.py`).
Cloud database	Neon, PostgreSQL 18	Store users and asteroids, enforce constraints and limit what the `aegis_app` role can do.
Authentication and authorization	bcrypt, JWT, API keys	Check passwords, issue and verify access tokens, check partner keys, and decide what each caller may do.

The application code will be split into modules so that database operations are separate from the endpoint logic:

```
api/index.py              routes (endpoints) and error handlers
api/auth.py               login, password check, tokens, partner keys, access rules
api/db.py                 connection and all SQL queries
api/models.py             Pydantic models for requests and responses
db/schema.sql             tables, constraints, indexes and grants
scripts/import_asteroids.py   one-time import of the catalog
scripts/create_user.py        creates a login account
frontend/                 website with the login screen
```

4.2 Request and Data Flow

Every request goes through the same steps: the route receives it, access control identifies the caller, FastAPI and Pydantic validate the input, the data access layer runs one or more SQL statements, and the route returns JSON with a status code. All values are sent to the database as query parameters and are never inserted into the SQL text, which prevents SQL injection.

Table 12. Access rules by caller

Endpoint	Guest (no login)	Logged-in user	Partner site key
GET `/`, GET `/health`	Allowed	Allowed	Allowed
POST `/api/v1/auth/login`	Allowed	Allowed	Allowed
GET `/api/v1/auth/me`	401	Allowed	401
GET `/api/v1/asteroids`	Featured records only (10)	All records	All records
GET `/api/v1/asteroids/random`	Featured records only	All records	All records
GET `/api/v1/asteroids/{id}`	Featured records only; others 401	All records	All records
GET `/api/v1/stats`	401	Allowed	Allowed
POST, PATCH, DELETE `/api/v1/asteroids…`	401	Allowed	403 (read-only key)

Requesting a List of Records

1. The client sends `GET /api/v1/asteroids` with optional filters: `search` (name or ID), `hazardous`, `min_diameter`, `max_diameter`, `from_date`, `to_date`, `sort`, `order`, `limit` and `offset`.
2. Access control identifies the caller. With no credentials the caller is a guest. An invalid or expired token returns 401 instead of falling back to guest, so the website knows the user must log in again.
3. FastAPI checks the type and range of every parameter and returns 422 if one is invalid.
4. The data access layer builds one SELECT with a condition for each filter given. For guests it adds `is_featured = true` and limits the result to 10 rows. A second value, the number of matching rows, is returned with `count(*) over ()`.
5. Neon returns the rows, and the API answers 200 with the same fields as version 1.1 (`count`, `matched`, `total`, `offset`, `limit`, `asteroids`) plus `access` ("guest" or "full").

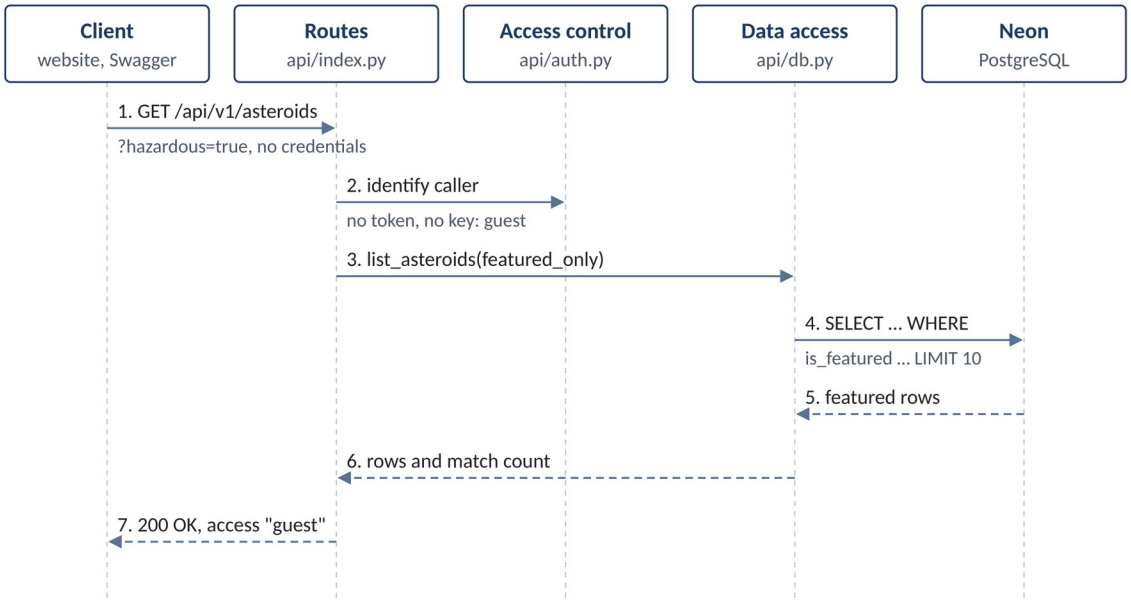


Figure 14. `flow-guest-list`: a guest requests the list of asteroids

Requesting an Individual Record

1. The client sends `GET /api/v1/asteroids/{neo_reference_id}`.
2. The API runs `SELECT … WHERE neo_reference_id = %s`, a primary key lookup.
3. If no row is found, the API returns 404 "Asteroid not found."
4. If the caller is a guest and the asteroid is not featured, the API returns 401 "Log in to view this asteroid."
5. Otherwise it returns 200 with the record.

Submitting a New Record

1. The client sends `POST /api/v1/asteroids` with a JSON body and the header `Authorization: Bearer <token>`.
2. Access control verifies the token. Missing or invalid: 401. Partner key: 403.
3. The body is parsed and validated against the create model. Malformed JSON: 400. Missing fields, wrong types or values out of range: 422 with the name of each invalid field.

4. The data access layer runs `INSERT … RETURNING *` with `created_by` set to the logged-in user. If 10 asteroids are already featured and the new one is marked as featured, the API returns 409.
5. If the ID or name already exists, PostgreSQL raises error 23505 and the API returns 409 with the field that is duplicated.
6. On success the API returns 201 Created with the stored record.

Updating an Existing Record

1. The client sends `PATCH /api/v1/asteroids/{neo_reference_id}` with only the fields to change, and the access token.
2. Access control and validation run as for create. At least one field must be sent, and `neo_reference_id` cannot be changed.
3. The data access layer builds `UPDATE asteroids SET <only the submitted fields>, updated_at = now() WHERE neo_reference_id = %s RETURNING *`. Fields that were not submitted keep their values.
4. If no row was updated, the API returns 404. A duplicate name returns 409.
5. On success the API returns 200 with the updated record.

Deleting a Record

1. The client sends `DELETE /api/v1/asteroids/{neo_reference_id}` with the access token.
2. Access control verifies the token (401 or 403 as above).
3. The data access layer runs `DELETE … WHERE neo_reference_id = %s RETURNING neo_reference_id`.
4. If no row was deleted, the record did not exist and the API returns 404.
5. On success the API returns 200 with a confirmation message. Later requests for the same ID return 404.

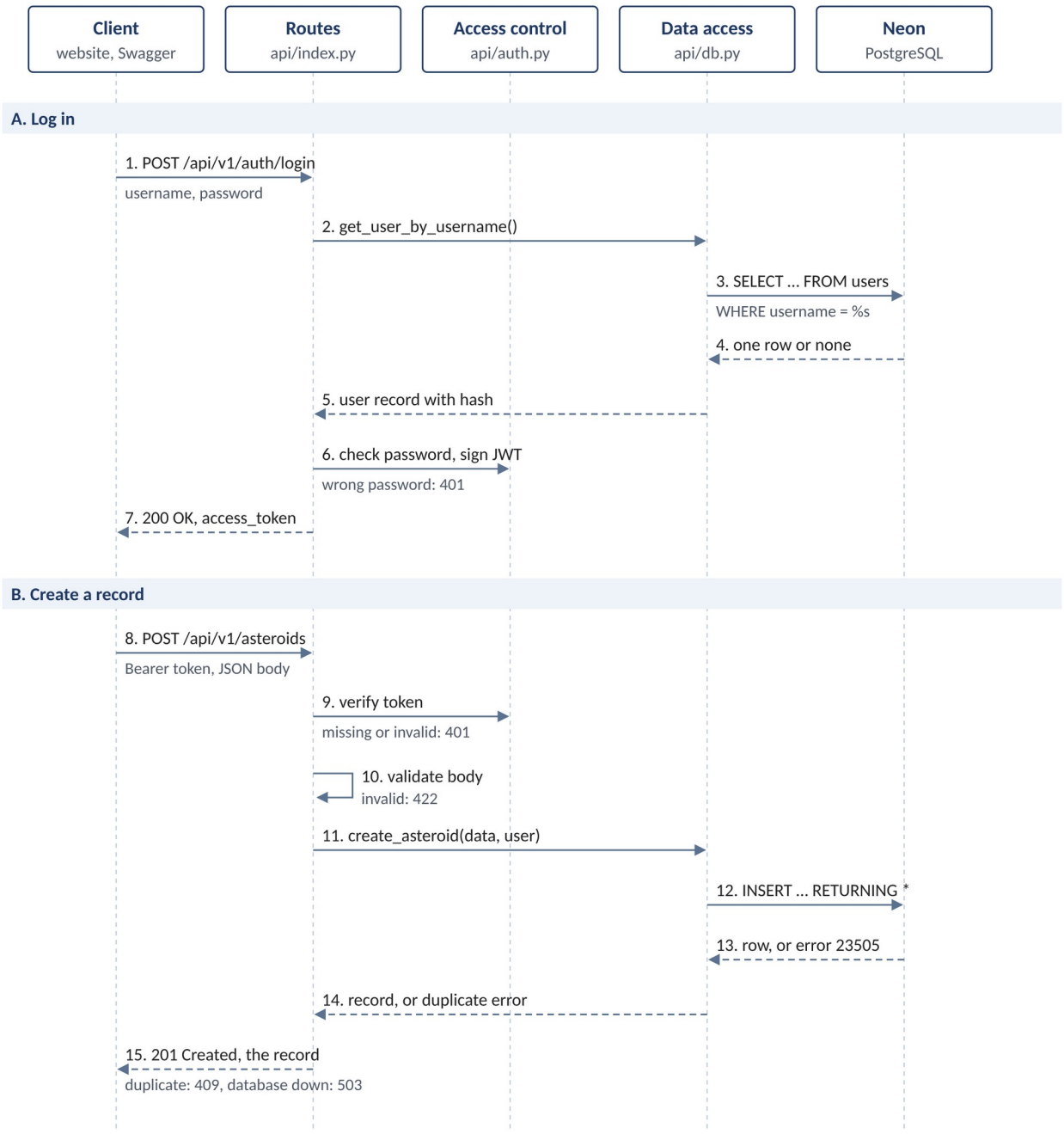


Figure 15. `flow-login-create`: a user logs in and creates a new asteroid

Planned Status Codes

Table 13. Planned HTTP status codes

Code	Meaning in the AegisNEO API
200 OK	Successful read, update, delete or login
201 Created	Record created
400 Bad Request	Body is not valid JSON

Code	Meaning in the AegisNEO API
401 Unauthorized	Login required, wrong username or password, or invalid or expired token
403 Forbidden	Valid partner key used for an operation that changes data
404 Not Found	No asteroid with this ID
409 Conflict	Duplicate ID or name, or more than 10 featured asteroids
422 Unprocessable Content	Missing fields, wrong types or values outside the allowed ranges
503 Service Unavailable	The database cannot be reached; no internal details are returned
500 Internal Server Error	Unexpected error; a generic message is returned and the details are logged on the server only

4.3 Integration Approach

Database Driver and Library

The API connects with **psycopg 3**, the PostgreSQL driver used in Part I (version 3.3.6). It sends plain SQL with parameters. An ORM such as SQLAlchemy is not used: the schema has only two tables, and writing the SQL directly keeps each query visible and easy to explain.

Required Dependencies

Table 14. Python dependencies

Package	Purpose	Status
fastapi 0.141.1	Web framework, routing, validation and Swagger UI	Existing
uvicorn 0.47.0	Local development server	Existing
pydantic 2.13.4	Request and response models	Existing
psycopg[binary] 3.3	PostgreSQL driver, with libpq included	New
PyJWT	Sign and verify access tokens	New
bcrypt	Hash and check passwords	New
python-multipart	Read the login form (required by FastAPI for form data)	New
pytest, httpx	Automated tests (development only)	New

Exact versions will be pinned in `requirements.txt` when they are installed in Phase 4.

Connection Method

- The API uses Neon's **pooled** connection string (host name ending in `-pooler`). The pooler keeps connections open to the database, so opening a connection from a short-lived serverless function is fast.
- TLS is required (`sslmode=require` and `channel_binding=require`). Part I confirmed that Neon refuses unencrypted connections.

- A connection is opened for each request and closed after the response, because Vercel can pause or stop a function instance between requests. In Part I a warm request took 22 to 30 ms, and the first request after the database had been idle for more than 5 minutes took about 1 second.
- Automatic prepared statements are turned off (`prepare_threshold=None`), because the pooler can send consecutive statements to different database sessions.
- `connect_timeout` is set to 10 seconds, so a database outage returns 503 quickly instead of waiting until Vercel stops the function.
- Each write runs in one transaction: committed on success and rolled back on any error.

Environment Variables

Table 15. Environment variables

Name	Used by	Where it is set	Format (no real values)
`DATABASE_URL`	API	Vercel; local `.env`	`postgresql://aegis_app:****@ep-…-pooler.ap-southeast-1.aws.neon.tech/neondb?sslmode=require&channel_binding=require`
`JWT_SECRET`	API	Vercel; local `.env`	Random 64-character hexadecimal value
`JWT_EXPIRE_MINUTES`	API (optional)	Vercel	Number of minutes; default 60
`AEGISNEO_API_KEYS`	API	Vercel	`impactor:****,neo-atlas:****` (partner sites only)
`DATABASE_OWNER_URL`	Setup scripts only	Developer's local `.env` only	Owner connection string. Never stored in Vercel.

A file `.env.example` with the variable names and placeholder values will be committed so another developer knows what to set. The real `.env` file is already excluded by `.gitignore` (`.env` and `.env.*`).

Additional Provider Configuration

- **Neon:** drop the Part I test table, run `db/schema.sql` as the owner (this enables the `pg_trgm` extension and grants the privileges to `aegis_app`), import the catalog, mark the 10 featured asteroids and create the demonstration accounts.
- **Vercel:** add the environment variables for the Production and Preview environments, and set the function region to Singapore (`"regions": ["sin1"]` in `vercel.json`). The default region is Washington, D.C. (`iad1`), which would add a trip across the Pacific to every database query, because the database is in Singapore.
- **Retired keys:** the demo key and the website key are removed from the code and are no longer accepted. They remain visible in the Git history, so removing them from the code alone would not be enough.

4.4 Security and Access Design

How Database Credentials Are Protected

- The connection string, which contains the `aegis_app` password, is stored only in Vercel environment variables and in the developer's local `.env` file. It is never written in the code, committed to GitHub, printed in logs or returned in a response.
- The owner credentials are never stored in Vercel. They are used only on the developer's machine to set up the database.
- Error responses never include the connection string, SQL text, driver messages or stack traces. Those details are written only to the server log.

How the API Authenticates Users and Requests

- **Users** log in with `POST /api/v1/auth/login` using a username and password, sent as form fields so that the Authorize button in Swagger UI works. The password is checked against the bcrypt hash stored in `users`.
- On success the API returns a JSON Web Token (JWT) signed with HS256 using `JWT_SECRET`. It contains the user ID, the username and an expiry time 60 minutes later. The client sends it in the header `Authorization: Bearer <token>`.
- A failed login always returns the same message, "Incorrect username or password.", whether the username exists or not, so the API does not reveal which usernames exist.
- The website keeps the token in `sessionStorage`, so it is removed when the browser tab is closed. The Log out button removes it immediately.
- **Partner sites** send their secret key in the `x-api-key` header. The key is compared with the values in `AEGISNEO_API_KEYS` using a constant-time comparison.
- **Guests** send no credentials.

Which Operations Require Authorization

Table 12 lists the rules. In summary, a guest can read only the 10 featured asteroids, a partner site can read everything but change nothing, and only a logged-in user can create, update or delete records and read the statistics. Database credentials are never used as a substitute for user login: every user logs in to the API, and only the API itself holds the database credentials.

How the Database Restricts Access

- The API connects as `aegis_app`, which can only read and write rows in `asteroids` and read `users` (Table 9). It cannot change the structure of the database or create accounts.
- Neon accepts only encrypted connections with the correct password. The Free plan has no IP allow-list, so the connection string must stay secret (Part I, Section 6.4).
- The constraints in Section 3.4 reject invalid data, even if a request bypassed the API's own validation.

Connection Type

The API uses a **server-side connection**. The browser and the partner sites never receive database credentials and never connect to Neon. All access goes through the API, where the login and access rules are applied. Neon's optional Data API (a REST interface to the database) is not enabled.

Known Limitations of the Design

- A token cannot be cancelled before it expires; logging out removes it only from the browser. The 60-minute expiry limits this risk.
- Repeated login attempts are not limited by the API. A rate-limit rule in the Vercel Firewall will be evaluated in Phase 6.
- The limit of 10 featured asteroids is checked by the API, not by a database constraint.

4.5 Required Output of Phase 3

Table 16. Phase 3 required output

Required by the activity	Where it is answered
System architecture diagram	Figure 13 and Table 11
Request and data flow	Section 4.2, Figures 14 and 15
Integration approach	Section 4.3
Security and access design	Section 4.4

5. References

[N4] Neon Docs: Connection pooling. https://neon.com/docs/connect/connection-pooling

[N6] Neon Docs: Neon plans. https://neon.com/docs/introduction/plans

[N8] Neon Docs: Manage database access. https://neon.com/docs/manage/database-access

[P1] PostgreSQL 18 Documentation: Constraints. https://www.postgresql.org/docs/18/ddl-constraints.html

[P2] PostgreSQL 18 Documentation: Numeric types (special floating-point values). https://www.postgresql.org/docs/18/datatype-numeric.html

[P3] PostgreSQL 18 Documentation: pg_trgm. https://www.postgresql.org/docs/18/pgtrgm.html

[P4] PostgreSQL 18 Documentation: Indexes. https://www.postgresql.org/docs/18/indexes.html

[Y1] Psycopg 3 Documentation: Prepared statements. https://www.psycopg.org/psycopg3/docs/advanced/prepare.html

[Y2] Psycopg 3 Documentation: Passing parameters to SQL queries. https://www.psycopg.org/psycopg3/docs/basic/params.html

[A1] FastAPI Documentation: OAuth2 with password (and hashing), Bearer with JWT tokens. https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/

[V2] Vercel Docs: Configuring regions for Vercel Functions. https://vercel.com/docs/functions/configuring-functions/region

[V3] Vercel Docs: Environment variables. https://vercel.com/docs/environment-variables

[O1] OWASP Cheat Sheet Series: Password Storage. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[O2] OWASP Cheat Sheet Series: Authentication. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

[K1] Kaggle: Nearest Earth Objects (1910-2024). https://www.kaggle.com/datasets/ivansher/nasa-nearest-earth-objects-1910-2024

[R1] Latagan, J. J. F. Cloud Database Investigation, Part I: Comparative Cloud Database Investigation. October 2026.